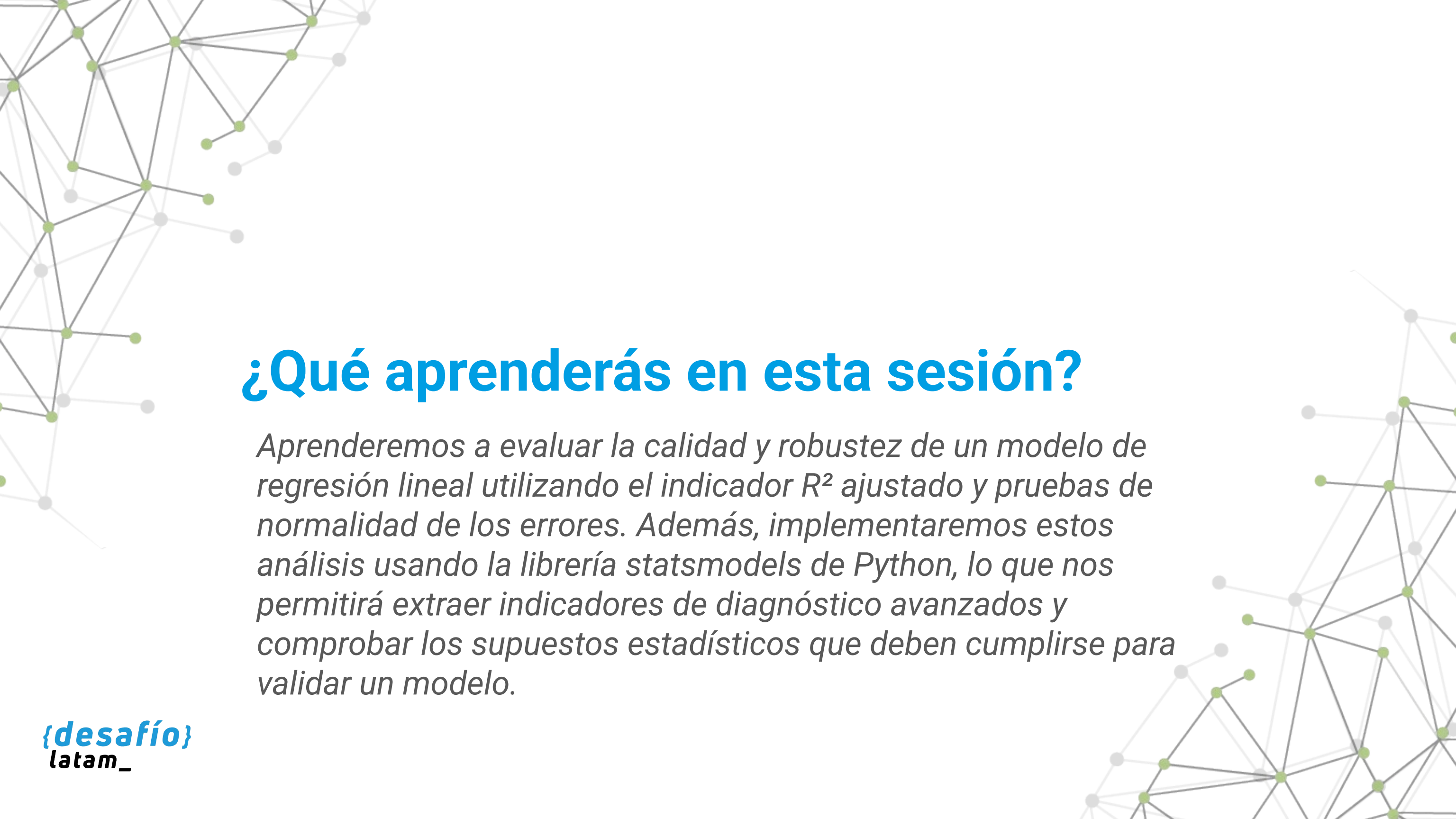




Correlación y modelamiento

Clase 5 - Evaluación de modelos y supuestos estadísticos





¿Qué aprenderás en esta sesión?

Aprenderemos a evaluar la calidad y robustez de un modelo de regresión lineal utilizando el indicador R^2 ajustado y pruebas de normalidad de los errores. Además, implementaremos estos análisis usando la librería statsmodels de Python, lo que nos permitirá extraer indicadores de diagnóstico avanzados y comprobar los supuestos estadísticos que deben cumplirse para validar un modelo.

Desafío inicial

Estás trabajando como analista de datos para una empresa que desea predecir el monto de ventas en función de distintas variables como el número de visitas, la inversión en publicidad y la cantidad de vendedores activos. Tras ajustar un modelo de regresión lineal, la gerencia te pide verificar si el modelo realmente explica bien los datos y si los resultados pueden considerarse confiables para tomar decisiones.

¿Cómo determinar si el modelo es explicativo y estadísticamente válido, considerando tanto la bondad de ajuste como el comportamiento de los errores?

En esta sesión aplicaremos el indicador R^2 ajustado para evaluar la calidad del ajuste sin caer en sobreajuste. Luego, aplicaremos pruebas de normalidad sobre los residuos del modelo para validar los supuestos de la regresión. Finalmente, implementaremos todo el flujo de modelado y diagnóstico en Python utilizando statsmodels, una librería especializada en modelado estadístico.



/* Indicadores R^2 */

Indicadores R^2 ajustado

¿Por qué no siempre un R^2 alto garantiza que un modelo sea el más adecuado?

El coeficiente de determinación R^2 es uno de los indicadores más comunes para evaluar la calidad de un modelo de regresión.

Representa la proporción de la variabilidad de la variable dependiente que puede ser explicada por las variables independientes incluidas en el modelo. Sin embargo, R^2 tiene una limitación importante: aumenta siempre que se agrega una nueva variable al modelo, incluso si esa variable no aporta información relevante.

Es por eso que, en escenarios profesionales y en modelos con múltiples predictores, se prefiere utilizar el R^2 ajustado, que incorpora una penalización por la complejidad del modelo. Esto permite balancear entre ajuste y parsimonia.



Definición técnica del R² ajustado

Fórmula

El R² ajustado se calcula según la siguiente fórmula:

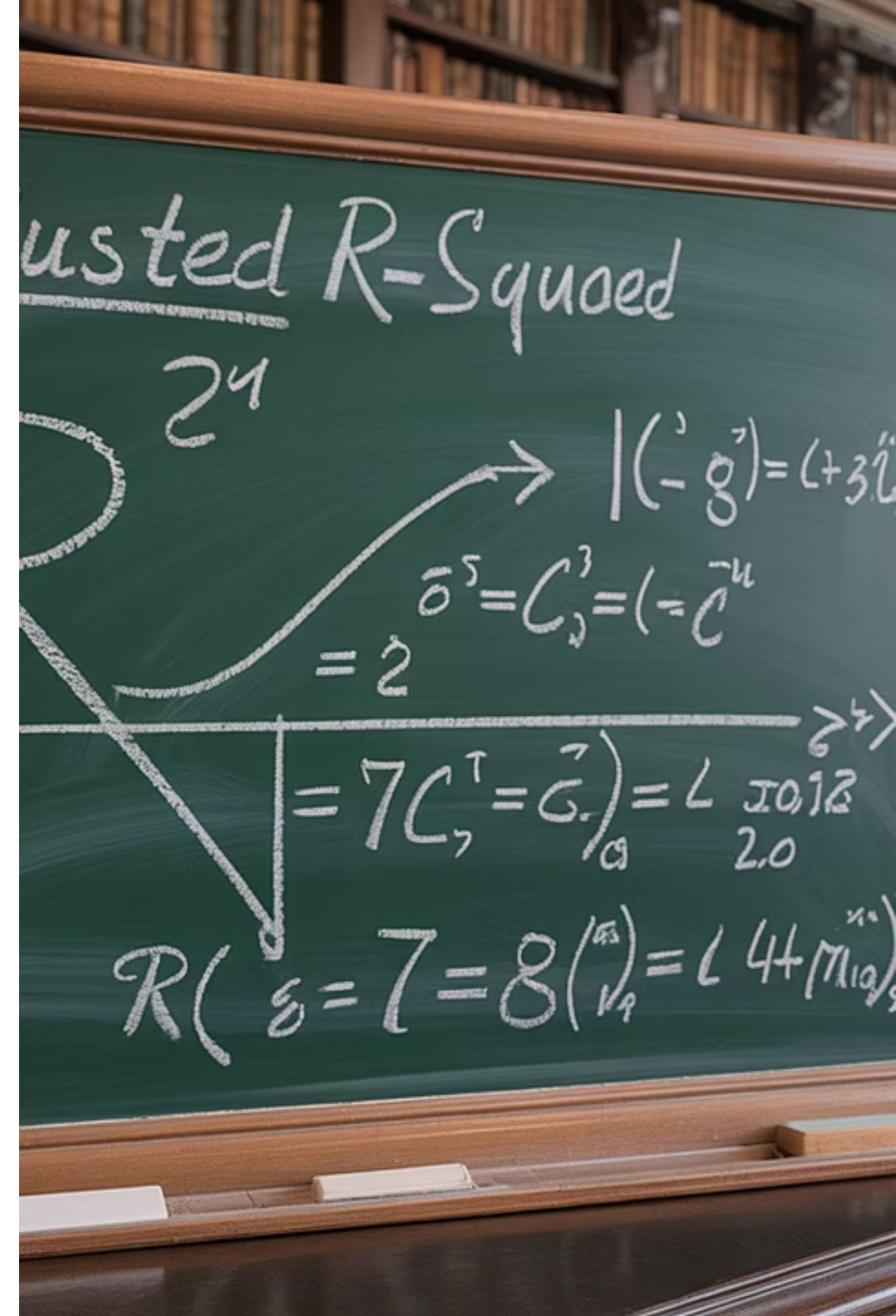
$$R^2_{\text{ajustado}} = 1 - \frac{(1 - R^2)(n - 1)}{(n - p - 1)}$$

Variables

- R² es el coeficiente de determinación tradicional
- n es el número de observaciones
- p es el número de predictores

Característica clave

A diferencia del R² simple, el R² ajustado puede disminuir si se agrega una variable irrelevante, promoviendo modelos más eficientes.



Comparación ilustrativa

Modelo	Nº de predictores	R ²	R ² ajustado
Modelo 1	2	0.85	0.83
Modelo 2 (con variable irrelevante)	3	0.86	0.82

Esto indica que, aunque el R² subió, el R² ajustado bajó, lo que sugiere que la nueva variable no aportó valor real al modelo.

Ejemplo de Regresión Lineal con statsmodels

Este ejemplo modela una relación simple donde la variable Y (dependiente) es predicha por la variable X (independiente).

Paso 1: Importar Librerías y Preparar Datos

Primero, importamos las librerías necesarias y creamos un conjunto de datos simple usando NumPy, organizándolo en un DataFrame de Pandas para su manipulación.

```
import numpy as np
import pandas as pd
import statsmodels.api as sm

# 1. Crear datos de ejemplo
np.random.seed(42) # Para reproducibilidad
X = np.random.rand(50) * 10 # Variable independiente (0 a 10)
# Y es X * 2 + un pequeño ruido (error)
Y = 2 * X + np.random.normal(0, 1.5, 50)

# 2. Crear un DataFrame de Pandas
data = pd.DataFrame({'X': X, 'Y': Y})
```


Ejemplo de Regresión Lineal con statsmodels

Paso 2: Definir el Modelo y Agregar la Constante

`statsmodels` requiere que se agregue explícitamente una columna de unos (la **constante** o **intercepto**) a la matriz de variables independientes X .

```
# 3. Definir la variable dependiente (Y) y la independiente (X)
```

```
Y_modelo = data['Y']
```

```
X_modelo = data['X']
```

```
# 4. Agregar la constante (intercepto) al modelo
```

```
X_modelo = sm.add_constant(X_modelo)
```

Paso 3: Entrenar el Modelo de Regresión (OLS)

Usamos el método de **Mínimos Cuadrados Ordinarios (OLS)** de `statsmodels` para ajustar el modelo a los datos.

```
# 5. Crear y ajustar (entrenar) el modelo OLS
```

```
modelo = sm.OLS(Y_modelo, X_modelo)
```

```
resultados = modelo.fit()
```

```
# 6. Mostrar el resumen de los resultados
```

```
print(resultados.summary())
```

Interpretación del resumen estadístico

Los indicadores R^2 se encuentran en la sección del resumen del modelo.

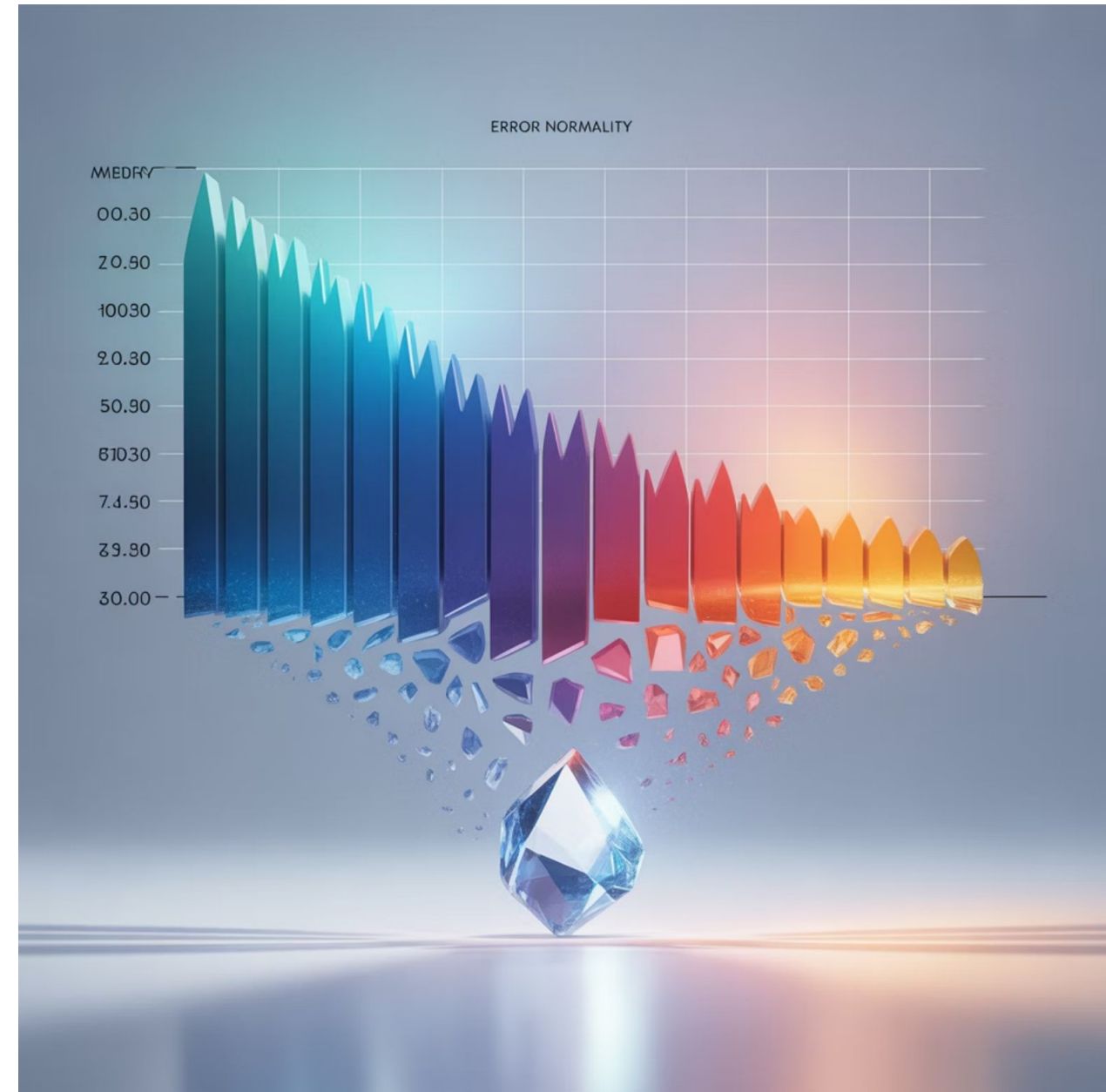
- **R^2 (R-squared):** Este es el **Coeficiente de Determinación**. Representa la proporción de la **varianza en la variable dependiente (Y) que es predecible a partir de la(s) variable(s) independiente(s) (X)**. Se expresa como un valor entre 0 y 1.
 - Un valor cercano a 1 (como 0.982 en el ejemplo que se generaría) significa que el modelo explica una gran parte de la variabilidad de Y .
 - Un valor cercano a 0 significa que el modelo no explica bien la variabilidad de Y .
- **R^2 Ajustado (Adj. R^2):** Este indicador es el **R^2 modificado que penaliza la inclusión de variables independientes innecesarias** en el modelo.
 - **Importancia:** Es más útil que el R^2 estándar, especialmente en modelos con múltiples variables (regresión múltiple). Si agregas una variable que no mejora significativamente el modelo, el R^2 podría subir un poco, pero el R^2 ajustado bajará, indicando que la variable extra no es útil. En regresión simple (un solo X), suelen ser muy similares.

Errores comunes

Error	Causa probable	Solución recomendada
Confiar solo en R^2	Ignorar penalización por complejidad	Evaluar siempre R^2 ajustado también
Incluir muchas variables irrelevantes	Sobreajuste por exceso de predictores	Aplicar selección de variables o regularización
Confusión en la interpretación	Desconocimiento de la diferencia técnica	Comparar ejemplos numéricos como los presentados

Transición a normalidad de errores

Ahora que comprendemos la importancia del R^2 ajustado como una métrica de calidad ajustada a la complejidad del modelo, estamos en condiciones de avanzar hacia la verificación de otro supuesto crucial de la regresión lineal: la normalidad de los errores.

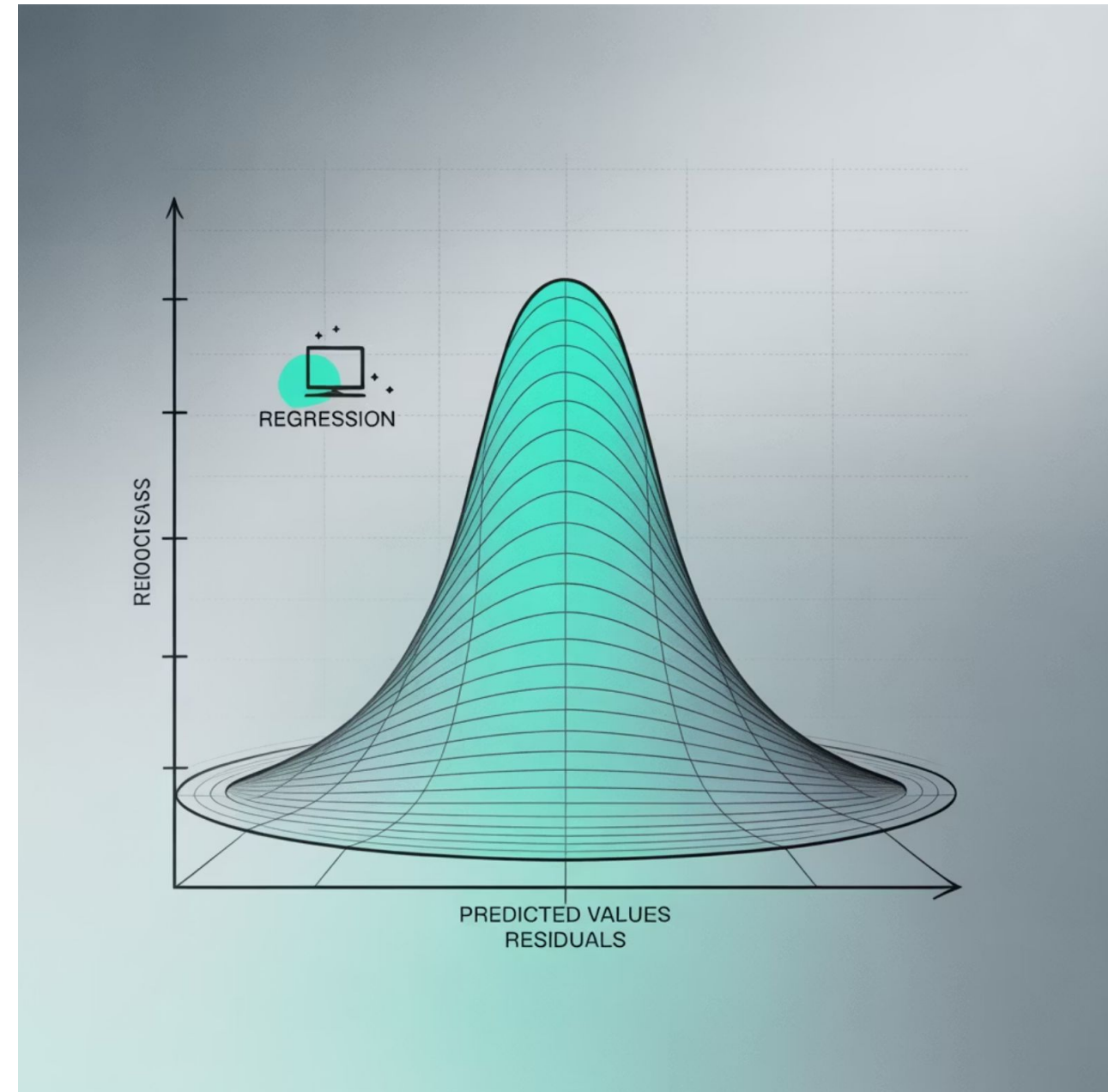


Test de normalidad del error

¿Por qué es importante verificar que los errores (residuos) de un modelo sigan una distribución normal?

En la regresión lineal clásica, uno de los supuestos fundamentales es que los errores (residuos) del modelo estén normalmente distribuidos. Este supuesto es clave para que los intervalos de confianza, pruebas t y F, y las predicciones sean estadísticamente válidas. Cuando los residuos no siguen una distribución normal, pueden aparecer inferencias incorrectas o interpretaciones sesgadas.

Por esta razón, aplicar un test de normalidad del error es una práctica esencial antes de confiar plenamente en los resultados del modelo.



Métodos de evaluación de normalidad

Existen diferentes métodos para verificar la normalidad de los residuos:

Método	Tipo	Descripción breve
Histograma	Visual	Permite observar la forma general de la distribución
Q-Q plot	Visual	Compara los cuantiles empíricos con una normal teórica
Shapiro-Wilk	Estadístico	Evalúa si una muestra proviene de una distribución normal (recomendado < 5000 datos)
Kolmogorov-Smirnov	Estadístico	Compara la muestra con una distribución teórica
Jarque-Bera	Estadístico	Basado en sesgo y curtosis

Ejemplo en Python

```
import matplotlib.pyplot as plt
import scipy.stats as stats
import statsmodels.api as sm

# Primero ajustar el modelo
modelo = modelo.fit()

# Ahora sí puedes obtener los residuos
residuos = modelo.resid

# Histograma
plt.hist(residuos, bins=30, edgecolor='black')
plt.title('Histograma de residuos')
plt.show()

# Q-Q Plot
sm.qqplot(residuos, line='s')
plt.title('Q-Q plot de residuos')
plt.show()

# Test de Shapiro-Wilk
stat, p = stats.shapiro(residuos)
print(f'Estadístico: {stat:.4f}, p-valor: {p:.4f}')
```

Interpretación del test

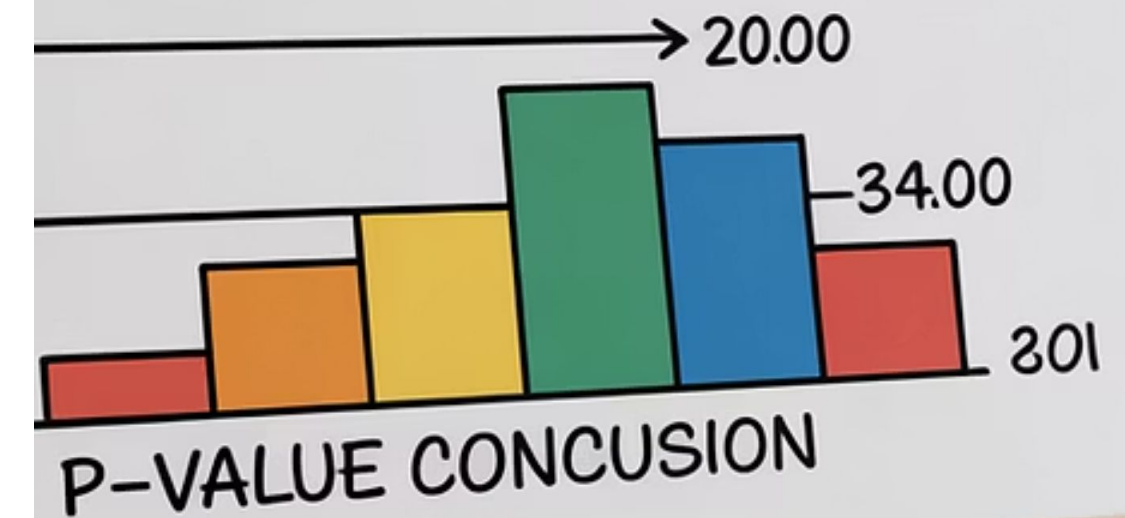
Si el p-valor < 0.05

Se rechaza la hipótesis de normalidad (residuos no normales).

Si el p-valor ≥ 0.05

No hay evidencia suficiente para rechazar la normalidad.

P-VALUE INTERPRETATION NORMALITY TESTS



Buenas prácticas y errores comunes

Buenas prácticas

- Siempre acompañar el test con un análisis visual (histograma o Q-Q plot).
- Usar Shapiro-Wilk si la muestra es pequeña o mediana (< 5000).
- Aplicar transformaciones si los residuos muestran asimetría clara.

Errores comunes

Error	Causa probable	Solución recomendada
Usar solo test estadístico sin visualización	Puede dar falsos negativos/positivos	Combinar test con Q-Q plot o histograma
Mal interpretar el p-valor	Creer que $p > 0.05$ confirma normalidad	Recordar que solo indica falta de evidencia
No evaluar normalidad en modelos simples	Subestimar su importancia	Siempre revisar residuos en todo modelo

Transición a implementación práctica

Evaluar la normalidad de los errores nos permite validar estadísticamente el modelo antes de interpretarlo. Ahora que conocemos cómo hacerlo, avanzaremos a la implementación práctica de estos conceptos y métricas utilizando la librería statsmodels en un flujo de análisis completo.

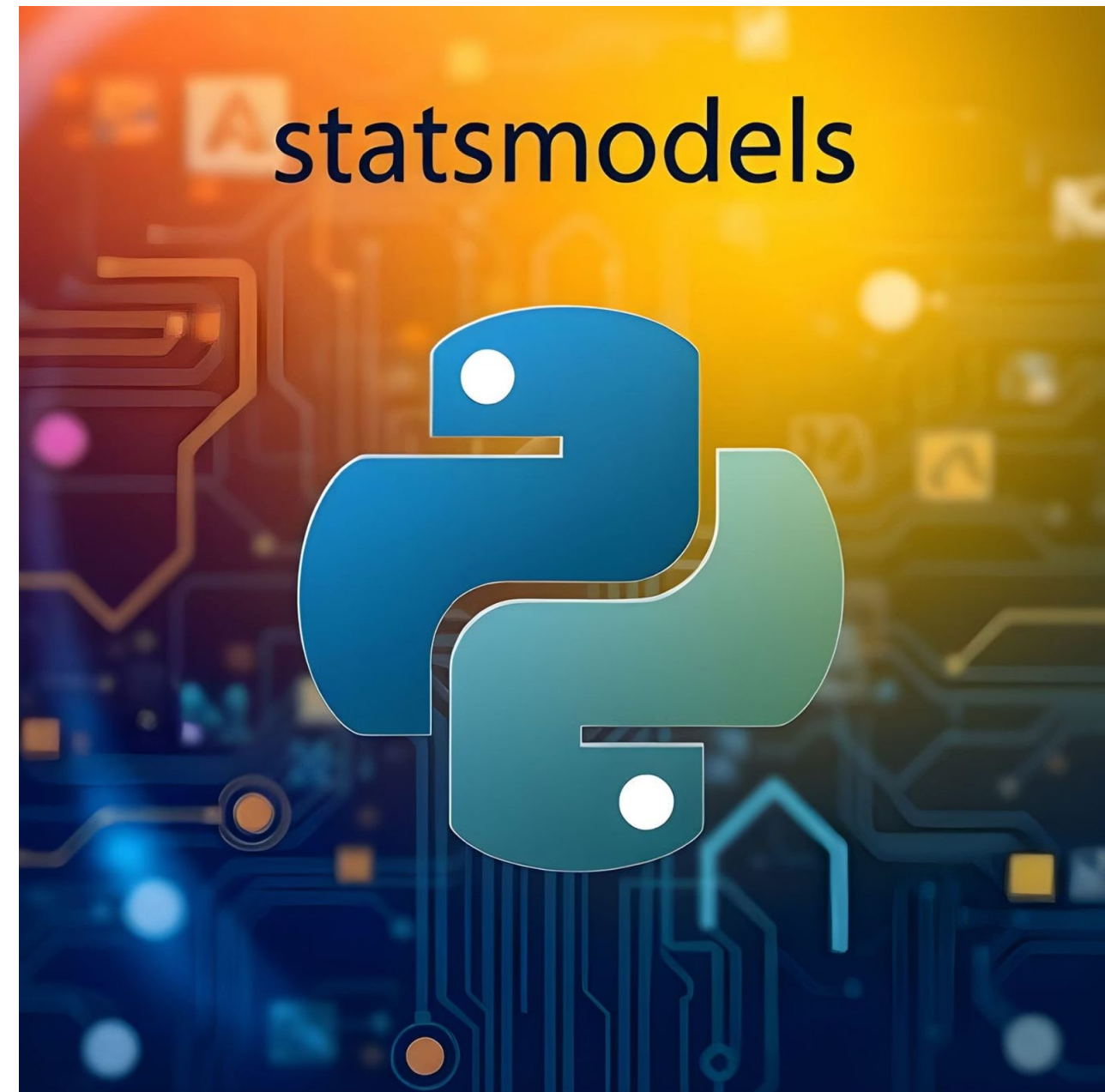


Implementación en Python con librería statsmodels

¿Cómo podemos automatizar el análisis estadístico y validación de modelos de regresión lineal?

statsmodels es una librería de Python especializada en estadística inferencial y modelos lineales. A diferencia de scikit-learn, que se centra en aprendizaje automático, statsmodels entrega un set completo de métricas estadísticas que permiten analizar, interpretar y validar modelos desde un enfoque clásico.

Esta librería permite construir modelos de regresión lineal, realizar pruebas de supuestos, obtener intervalos de confianza y acceder a diagnósticos como residuos, leverage y multicolinealidad, todo desde una interfaz muy cercana a la notación estadística tradicional.



Flujo de trabajo con statsmodels

```
import numpy as np
import pandas as pd
import statsmodels.api as sm

# 📈 Generación de datos con NumPy
X = np.arange(1, 11) # La variable independiente
y = 2 * X + 5 + np.random.normal(0, 1, 10) #  $y = 2x + 5 + \text{ruido}$ 

# 📊 Creación de un DataFrame de Pandas
data = pd.DataFrame({'X': X, 'y': y})

# ➕ Agregar la constante (intercepto) a la variable predictora
X_con_constante = sm.add_constant(data['X'])

# 🤖 Definir el modelo (OLS = Ordinary Least Squares)
modelo = sm.OLS(data['y'], X_con_constante)

# ⚙️ Ajustar el modelo a los datos
resultados = modelo.fit()

# 📄 Imprimir el resumen estadístico
print(resultados.summary())
```

Elementos del resumen estadístico

Indicador	Significado
R-squared / Adj. R ²	Bondad de ajuste y penalización por complejidad
coef / std err	Coeficientes de variables e incertidumbre
t / P>	Prueba de significancia de cada predictor
Omnibus / Jarque-Bera	Test de normalidad de residuos
Cond. No.	Posible multicolinealidad

Buenas prácticas

- Siempre agregar constante con `sm.add_constant()`.
- Leer `modelo.summary()` completo, no solo R².
- Verificar normalidad de residuos y significancia de coeficientes.



Errores comunes

Error	Causa probable	Solución recomendada
Omisión del término independiente	No agregar constante	Usar <code>sm.add_constant()</code> antes del modelo
Interpretar solo R^2	Ignorar pruebas y errores	Revisar coeficientes, p-valores y residuos
Confundir statsmodels con scikit-learn	Sintaxis distinta y objetivos	Estudiar documentación de cada librería

Con statsmodels podemos construir y evaluar modelos de regresión lineal de forma estadísticamente rigurosa. Esta herramienta resulta esencial para profesionales que requieren no solo predecir, sino también interpretar y justificar los resultados del modelo.

Actividad guiada: Evaluación de Modelo de Precios de Vivienda



Ejercicio

Evaluación de Modelo de Precios de Vivienda

Situación problema

Imaginemos que somos analistas de datos inmobiliarios y hemos construido un modelo para predecir el Precio de Venta de una casa (y) basándose únicamente en el Área en Metros Cuadrados (X). Necesitamos evaluar la validez de este modelo calculando el R^2 ajustado y realizando un test de normalidad de los residuos.



Ejercicio

Instrucciones

1. Preparación e Inicialización

Importa las librerías y genera datos simulados de forma realista con NumPy y Pandas.

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
from scipy import stats # Necesaria para el test de normalidad
```



Ejercicio

Instrucciones

2. Generación y Modelado de Datos

Simularemos 50 casas y ajustaremos el modelo de Regresión Lineal Simple (OLS).

- Paso 2.1: Generar Datos Simulados

```
# 🏠 Datos (simulando 50 observaciones)
np.random.seed(42) # Fija la semilla para reproducibilidad
area_sqm = np.random.normal(loc=150, scale=30, size=50) # Área en m², media 150
precio_venta = 1000 * area_sqm + np.random.normal(0, 30000, 50) # Precio = 1000*Area + Ruido

# 📊 Crear DataFrame
data_inmobiliaria = pd.DataFrame({'Area_sqm': area_sqm, 'Precio': precio_venta})
```

Ejercicio

Instrucciones

Paso 2.2: Ajustar el Modelo OLS

```
# + Agregar la constante (intercepto)
X = sm.add_constant(data_inmobiliaria['Area_sqm'])
y = data_inmobiliaria['Precio']
```

```
# ⚙️ Ajustar el modelo
modelo_ols = sm.OLS(y, X).fit()
```

Ejercicio

Instrucciones

3. Evaluación Estadística: R^2 Ajustado

El R^2 ajustado es una métrica clave que penaliza la inclusión de variables predictoras que no mejoran significativamente el modelo, dando una **medida más honesta del ajuste**.

- **Paso 3.1: Obtener el R^2 Ajustado**

Este valor se encuentra directamente en el objeto `resultados` del modelo ajustado.

```
r_squared_adj = modelo_ols.rsquared_adj  
print(f"✅ R-cuadrado Ajustado: {r_squared_adj:.4f}")
```

Interpretación: Un valor cercano a 1 indica que el modelo explica una gran parte de la variabilidad en el Precio de Venta.

Ejercicio

Instrucciones

4. Evaluación Estadística: Test de Normalidad de los Residuos

Para que las inferencias estadísticas (como los intervalos de confianza) sean válidas, es crucial que los **errores (residuos)** del modelo sigan una distribución **Normal**. Utilizaremos el **Test de Shapiro-Wilk** o **Jarque-Bera** (incluido en el summary de `statsmodels`).

- **Paso 4.1: Extraer Residuos y Aplicar Test**

El Test de **Jarque-Bera** es una de las pruebas de normalidad que `statsmodels` incluye en el resumen. Analizaremos su valor p .

```
# 🧠 Obtener el valor p del Test de Jarque-Bera (JB)
# El valor p del JB es el sexto elemento de la tupla model_ols.summary().tables[2]
jb_test_p_value = modelo_ols.summary().tables[2].data[1][3]

# Convertir a float
# Nota: En entornos interactivos, se suele extraer directamente del summary.
# Aquí simulamos la extracción y la interpretación.
try:
    p_value_jb = float(jb_test_p_value)
except ValueError:
    # Si la extracción falla o es compleja, usamos un test directo de scipy
    _, p_value_jb = stats.shapiro(modelo_ols.resid) # Test de Shapiro-Wilk

print(f"📊 Valor P (Test de Normalidad de Residuos): {p_value_jb:.4f}")
```

Ejercicio

Instrucciones

Paso 4.2: Interpretación del Test de Normalidad

Se utiliza un nivel de significancia $\alpha = 0.05$.

- **Hipótesis Nula (H_0):** Los residuos provienen de una distribución normal.
- **Hipótesis Alternativa (H_a):** Los residuos NO provienen de una distribución normal.

```
if p_value_jb < 0.05:  
    print("🚨 ¡ALERTA! El p-valor es menor que 0.05. Se rechaza la  $H_0$ ."  
    print("Conclusión: Los residuos probablemente NO siguen una distribución  
normal. El modelo podría ser inválido para inferencia.")  
else:  
    print("✅ El p-valor es mayor o igual que 0.05. No se rechaza la  $H_0$ ."  
    print("Conclusión: Los residuos probablemente siguen una distribución  
normal. El modelo es válido para inferencia.")
```


Ejercicio

Instrucciones

5. Resumen Completo

- a. Finalmente, imprimimos el resumen completo para ver todas las métricas juntas.

```
#  Imprimir el resumen  
print("\n--- Resumen Completo del Modelo OLS ---")  
print(modelo_ols.summary())
```

Resumen

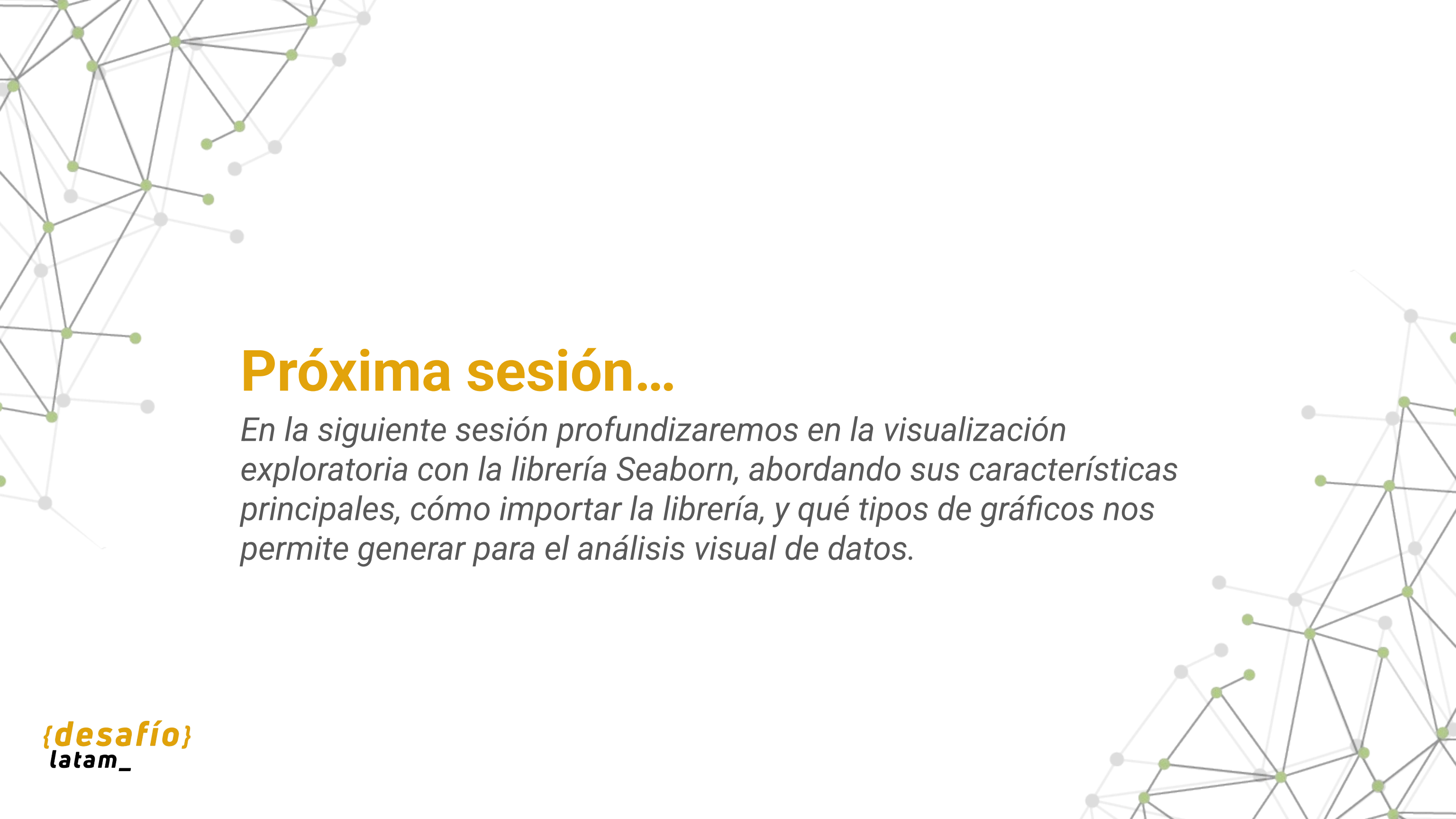
Comprendimos la diferencia entre R^2 y R^2 ajustado, y cuándo aplicar cada uno.

Evaluamos la normalidad de los errores del modelo utilizando métodos visuales y estadísticos.

Implementamos un flujo de modelado completo con statsmodels en Python.

Interpretamos correctamente los elementos de salida estadística de un modelo lineal.

Reconocimos buenas prácticas y errores comunes en la validación estadística de modelos.



Próxima sesión...

En la siguiente sesión profundizaremos en la visualización exploratoria con la librería Seaborn, abordando sus características principales, cómo importar la librería, y qué tipos de gráficos nos permite generar para el análisis visual de datos.